databricks

Building Lakeflow Spark Declarative Pipelines

LECTURE

Streaming Joins Overview

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

In this lecture, you will learn the fundamentals of streaming joins, including how to perform stream-snapshot joins, join streaming tables using materialized views, and execute stream-stream joins to combine real-time data from multiple sources.

Streaming Joins Overview

Stream-Snapshot Join Overview

Streaming Table to **Static Table**

The goal is to **incrementally** join new data with a **static table**

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

When performing joins with streaming tables in Declarative Pipelines, it's important to understand the different join types and how they behave.

Let's start with the most straightforward: Joining a Streaming Table to a Static Table (sometimes called a Stream-Snapshot Join or Stream-static join depending on the scenario)

In this pattern, we incrementally join new data from a streaming table to a static lookup table to create another streaming table.

For example, imagine you're ingesting a stream of transactions that includes a **country_code** column. You can join that with a static reference table that maps each code to a full country name.

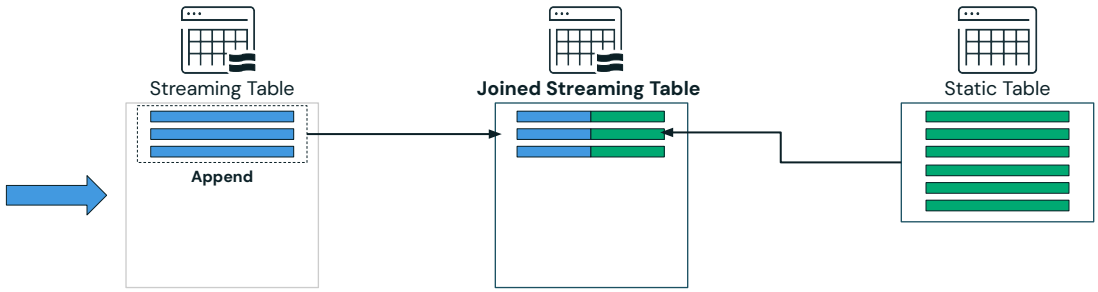
This is useful when you want to enrich your streaming data with reference information that doesn't change often.

Refer to: [Streaming tables | Databricks on AWS]

Streaming Joins Overview

Stream-Snapshot Join Overview

Streaming Table to **Static Table**



The **new data** in the streaming table JOINS using all of the **data in the lookup table**

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's walk through how this type of join actually works in practice.

As new data arrives in your streaming table, it is joined in real time with the static reference table.

That means every new record in the streaming table is matched against all available data in the static table, ensuring a complete and accurate join each time.

The result of that join is then appended to your final streaming output table, continuing the pipeline flow.

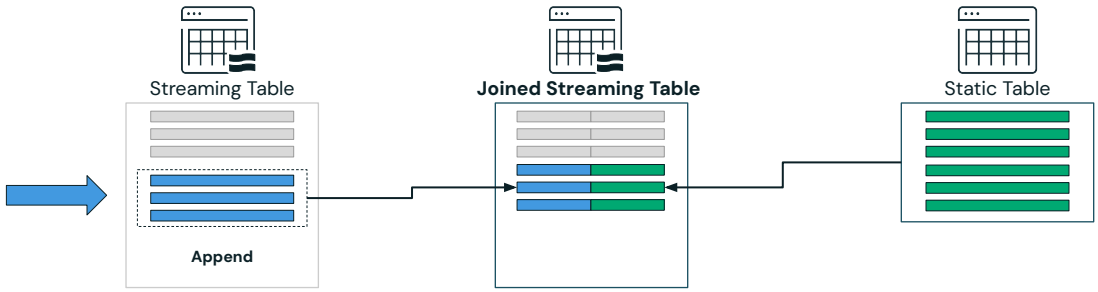
This is a great pattern for enriching streaming data with context, like converting country codes, product IDs, or user roles into readable formats using static reference data.

It's efficient, reliable, and works well since only one side of the join, the streaming table, is changing over time.

Streaming Joins Overview

Stream-Snapshot Join Overview

Streaming Table to **Static Table**



Only the new data in the streaming table JOINS using all of the **data in the lookup table**

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

A key detail to understand here is that as new data is appended to your source streaming table, only the new rows are joined with the entire static lookup table.

This is an incremental join, meaning the static table doesn't need to be reprocessed, only the incoming streaming data is evaluated.

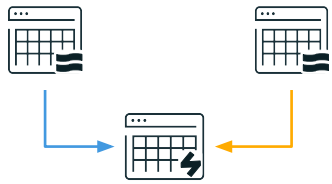
This keeps the process highly efficient, and ensures that each new record is immediately enriched with the latest reference data from the static table.

Streaming Joins Overview

Joining Streaming Tables with a Materialized View

Streaming Table to Static Table

Streaming Table to **Streaming Table** with a **Materialized View**



The goal is to take **all rows from two streaming tables** and join them together each time the pipeline is run

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Another supported pattern is joining two streaming tables using a materialized view.

The goal here is to join all rows from both streaming tables each time the pipeline is run.

This type of join is typically used when both input datasets are changing continuously, and you need to combine them regularly to produce a unified, up-to-date result.

For example, imagine one stream contains customer activity and another contains product catalog updates. You could join them using a materialized view to enrich activity data with the most recent product information.

Because both sides are streaming, a materialized view is required to handle this join efficiently and keep the results current.

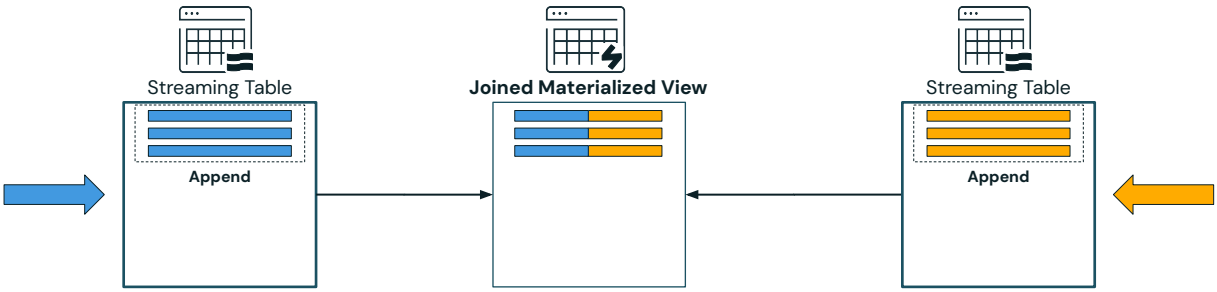
The materialized view will process all new rows from both tables and incrementally refresh, depending on pipeline configuration and compute mode.

Streaming Joins Overview

Joining Streaming Tables with a Materialized View

Streaming Table to Static Table

Streaming Table to **Streaming Table** with a **Materialized View**



The materialized view **efficiently computes all the data** in the streaming tables and joins the rows.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's walk through how this join actually works in practice.

As new data is appended to both streaming tables, the materialized view detects the changes and responds accordingly.

Each time the pipeline runs, the materialized view will efficiently compute the join by processing all the current data from both streaming tables.

This ensures that every new record from either side is matched correctly, producing an up-to-date and complete output.

The use of a materialized view allows the join to scale efficiently, leveraging incremental refresh where possible to avoid unnecessary recomputation.

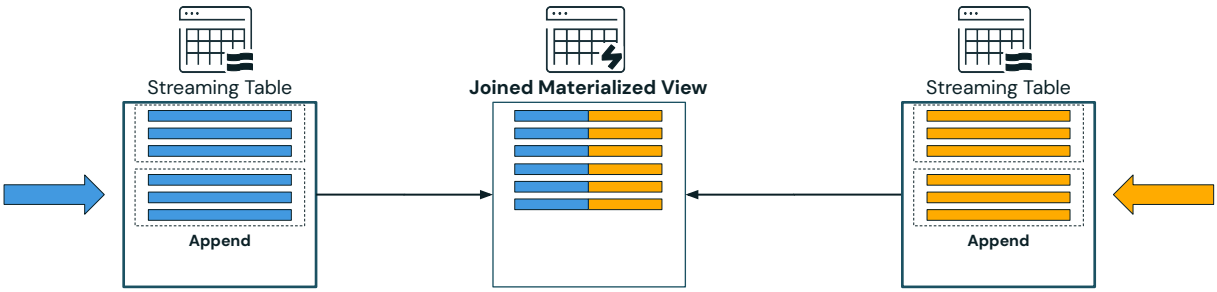
This join pattern is ideal when both data sources are live and frequently updated, and you want to keep your results synchronized across both.

Streaming Joins Overview

Joining Streaming Tables with a Materialized View

Streaming Table to Static Table

Streaming Table to **Streaming Table** with a **Materialized View**



As new data is added to the streaming tables, the materialized view will again **efficiently compute all the data** in the streaming tables and joins the rows

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

As new data is added to the streaming tables, the materialized view will again efficiently compute all the data in the streaming tables and joins the rows.

Streaming Joins Overview

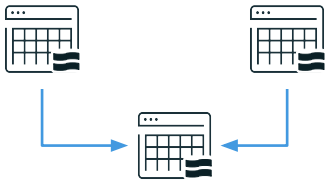
Stream-Stream Joins

Outside the scope of the course

Streaming Table to Static Table

Streaming Table to Streaming Table with a Materialized View

Incremental Stream to Stream Joins



The goal is to **incrementally** join new data from two tables, **past data is not used**

Databricks Streaming and DLT (Advanced) Course

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

The final type of streaming join is a stream-stream join, which is designed to incrementally join new data from two streaming tables as it arrives.

In this pattern, only the new incoming data from each stream is joined, past data is not considered during each run.

These joins are useful for detecting relationships between events that occur close together in time, such as joining clickstream data with real-time ad impressions.

However, because stream-stream joins often involve windowing logic, watermarking, and other advanced streaming concepts, they are outside the scope of this course.

We recommend reviewing additional Databricks documentation or advanced streaming resources if your use case requires this type of real-time event correlation.

Refer to:

- Work with joins on Databricks | Databricks on Google Cloud
- Optimize stateful processing in Lakeflow Declarative Pipelines with watermarks | Databricks on AWS

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.